

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – APPLICATION BASED QUESTIONS

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO1 – Precision (BL1, BL2, BL3)	Assessment:	Assessment Tool 1 – Application Based Questions
Weightage:	40% of CIA	Total Marks:	100 (2 Questions × 50 Marks)
CO1 Statement:	<i>Determine algorithm efficiency using asymptotic notations and mathematical techniques including Master Theorem and substitution method. (BL1, BL2, BL3)</i>		
Student Name:	H. Bhavani	Reg. No.:	192524385
Section / Batch:	AIDS	Date:	17/7/2026

Instructions to Students

- Answer BOTH questions. Each question carries 50 Marks. Total: 100 Marks.
- Clearly show all steps in derivations and complexity analysis.
- Use proper asymptotic notation (Big-O, Big- Θ , Big- Ω) wherever required.
- Justify each step in recurrence solving using appropriate methods.
- Neat presentation and logical explanation are mandatory

Question Type	Focus Area	Questions	Marks
Application-Based Questions	Apply Master Theorem and asymptotic analysis to real-world scenarios	Q1 – Q2	Q1 –50 Q2 –50
GRAND TOTAL:		100 Marks	

SECTION A – APPLICATION BASED QUESTIONS**Code of Conduct**

M. Ghafari (192524385) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: M. Ghafari

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20%	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30%	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20%	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20%	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10%	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Marks Evaluation:

Question No.	Understanding of Problem Context (20%)	Application of Concepts (30%)	Problem-Solving Approach (20%)	Accuracy of Solution (20%)	Clarity (10%)	Total Marks
Q1 (50 Marks)	3	3	3	3	3	38
Q2 (50 Marks)	3	3	3	3	3	38
Overall Total (100)						76

Faculty In-charge	Course Coordinator	Head of Department
Signature: <u>M. Ghafari</u>	Signature: _____	Signature: _____
Date: <u>17/7/2026</u>	Date: _____	Date: _____

Q7. Application: Search Engine Index Builder

A search engine builds an index using $T(n) = T(n/2) + 1$. Apply the substitution method to solve the recurrence. Expand the relation step-by-step and derive complexity. Analyze scalability as data grows. Interpret the result using asymptotic notation.

Q47. Application: Data Mining Pattern Extraction System

A data mining system processes large datasets using the recurrence $T(n) = 4T(n/4) + n \log n$. Apply the Master Theorem to determine the time complexity. Clearly identify parameters and justify the selected case. Analyze scalability for large data mining tasks. Interpret efficiency using asymptotic notation.

Q7. Application: Search Engine Index Builder

Question

A search engine builds an index using the recurrence:

$$T(n) = T(n/2) + 1$$

Apply the substitution method to solve the recurrence. Expand the relation step-by-step and derive complexity. Analyze scalability as data grows. Interpret the result using asymptotic notation.

Answer

Introduction

A search engine stores billions of web pages in an index to allow users to search information quickly. While building this index, the dataset is repeatedly divided into smaller parts until every page is processed. The recurrence

$$T(n) = T(n/2) + 1$$

represents this divide-and-process strategy. Here, each recursive call reduces the problem size by half, while performing a constant amount of work during each step.

Step 1: Given Recurrence

$$T(n) = T\left(\frac{n}{2}\right) + 1$$

Where,

- **T(n)** = Time required to build the index for **n** web pages.
- **T(n/2)** = Time taken after dividing the dataset into half.
- **1** = Constant work performed during each recursive call.

Step 2: Expand Using Substitution Method

- Substitute repeatedly.
First Expansion
- $T(n) = T(n/2) + 1$
- Substitute for $T(n/2)$
- $T(n) = [T(n/4) + 1] + 1$

Second Expansion

$$T(n) = T(n/8) + 3$$

Third Expansion

$$T(n) = T(n/16) + 4$$

General Expansion

After k substitutions,

$$T(n) = T\left(\frac{n}{2^k}\right) + k$$

Step 3: Find the Stopping Condition

The recursion stops when

$$\frac{n}{2^k} = 1$$

Therefore,

$$2^k = n$$

Taking logarithm,

$$k = \log_2 n$$

Step 4: Substitute Back

$$T(n) = T(1) + \log_2 n$$

Since

$$\begin{aligned} T(1) &= 1 \\ T(n) &= 1 + \log_2 n \end{aligned}$$

Ignoring constants,

$$T(n) = \Theta(\log n)$$

Time Complexity

Best Case

$$\Theta(\log n)$$

Average Case

$$\Theta(\log n)$$

Worst Case

$$\Theta(\log n)$$

Asymptotic Analysis

Big-O Notation

$O(\log n)$ Upper bound.

Big-Omega

$\Omega(\log n)$ Lower bound.

Big-Theta

$\Theta(\log n)$ Tight bound.

Scalability Analysis

The recurrence reduces the dataset by half during every recursive call.

Example:

Number of Pages Recursive Calls

16	4
32	5
64	6
128	7
256	8
512	9
1024	10

Even when the number of web pages doubles, only one extra recursive step is needed.

Hence, the algorithm scales efficiently for very large search engine datasets.

Real-World Interpretation

Google, Bing, and Yahoo process enormous datasets.

Instead of processing billions of pages at once, they divide them into smaller groups.

Each division reduces the work significantly.

This divide-and-conquer strategy keeps indexing fast and memory efficient.

Because the number of recursive calls grows slowly, the indexing system remains scalable.

Advantages

- Efficient for large datasets.
- Very low time complexity.
- Easy to implement.
- Suitable for divide-and-conquer algorithms.
- Highly scalable.

Conclusion

Using the substitution method,

$$T(n) = T(n/2) + 1$$

expands until the problem size becomes one.

The solution is

$$T(n) = 1 + \log n$$

Therefore,

$$T(n) = \Theta(\log n)$$

The search engine index builder grows logarithmically with the input size, making it highly efficient for indexing massive collections of web pages.

Q47. Application: Data Mining Pattern Extraction System

Question

A data mining system processes large datasets using the recurrence

$$T(n) = 4T(n/4) + n \log n$$

Apply the Master Theorem to determine the time complexity. Clearly identify parameters and justify the selected case. Analyze scalability for large data mining tasks. Interpret efficiency using asymptotic notation.

Answer

Introduction

Data mining extracts useful patterns, relationships, and knowledge from large datasets. Modern mining algorithms divide huge datasets into smaller subsets and process them recursively.

The recurrence

$$T(n) = 4T(n/4) + n \log n$$

describes this divide-and-conquer process.

Step 1: Identify Parameters

Compare with Master Theorem

$$T(n) = aT(n/b) + f(n)$$

Here,

- $a = 4$
- $b = 4$
- $f(n) = n \log n$

Step 2: Compute

Calculate

$$n^{\log_b a}$$

Since

$$\begin{aligned}\log_4 4 &= 1 \\ n^{\log_4 4} &= n\end{aligned}$$

Therefore,

$$n^{\log_b a} = n$$

Step 3: Compare Functions

We have

$$f(n) = n \log n$$

and

$$n^{\log_b a} = n$$

Observe

$$n \log n = n^{\log_b a} \log n$$

This matches

$$f(n) = \Theta(n^{\log_b a} \log^k n)$$

where

$$k = 1$$

Step 4: Apply Master Theorem

According to **Master Theorem Case 2**

If

$$f(n) = \Theta(n^{\log_b a} \log^k n)$$

then

$$T(n) = \Theta(n^{\log_b a} \log^{k+1} n)$$

Substitute values

$$T(n) = \Theta(n \log^2 n)$$

Final Time Complexity

$$\boxed{\Theta(n \log^2 n)}$$

Asymptotic Analysis

Big-O

$$O(n \log^2 n)$$

Upper bound.

Big-Omega

$$\Omega(n \log^2 n)$$

Lower bound.

Big-Theta

$$\Theta(n \log^2 n)$$

Exact complexity.

Scalability Analysis

The algorithm divides the dataset into four equal parts.

Each recursive level processes

$$n \log n$$

work.

Although the number of recursive calls increases, each subproblem becomes significantly smaller.

Example:

Dataset Size Approximate Growth

1,000	Low
10,000	Moderate
100,000	Higher
1,000,000	Efficient compared to quadratic algorithms

Growth is much slower than

$$O(n^2)$$

making it suitable for big data applications.

Real-World Application

Pattern extraction is widely used in:

- Customer purchase analysis
- Fraud detection
- Medical diagnosis
- Recommendation systems
- Social media analytics
- Banking and finance
- Market basket analysis

- Business intelligence

The divide-and-conquer strategy allows the system to process millions of records efficiently.

Advantages

- Faster than quadratic algorithms.
- Suitable for very large datasets.
- Efficient recursive decomposition.
- Good scalability.
- Used in modern machine learning and data mining systems.

Conclusion

Given the recurrence

$$T(n) = 4T(n/4) + n \log n$$

Using the Master Theorem:

- $a = 4$
- $b = 4$
- $f(n) = n \log n$

Since

$$f(n) = \Theta(n^{\log_b a} \log n)$$

Master Theorem **Case 2** applies.

Therefore,

$$T(n) = \Theta(n \log^2 n)$$

The data mining pattern extraction system scales efficiently for large datasets and is significantly more efficient than quadratic-time algorithms, making it well suited for big-data analytics and modern data mining applications.